Sample Knowledge Document for RAG Testing

Document ID: RAG-SAMPLE-2026

Purpose: This PDF is created as a clean, text-based sample document for testing a Retrieval-Augmented Generation Document Q&A; system. It contains structured information, page-level facts, policies, metrics, and implementation details so the system can demonstrate accurate source-grounded answers.

1. Project Overview

The RAG-Based Document Q&A; System allows a user to upload PDF documents and ask natural language questions. The system extracts text from the document, cleans the text, splits it into chunks, converts each chunk into embeddings, stores those embeddings in a vector database, retrieves the most relevant chunks for a query, and produces an answer based only on the retrieved context.

The system is designed for students, analysts, and engineering teams who need fast answers from long documents without manually reading every page. It is especially useful for resumes, academic reports, policy documents, product manuals, meeting notes, invoices, and research papers.

2. Key Objectives

- Build a modular Python project that follows clean engineering practices.

- Preserve metadata such as file name, page number, and chunk number for every text chunk.

- Return answers with confidence level and sources used.

- Avoid hallucination by answering only from uploaded document context.

- Provide both a Streamlit user interface and a FastAPI backend.

- Include pytest tests, logging, Docker support, and GitHub Actions workflow.

3. Success Criteria

The project is considered successful when a user can upload a text-based PDF, process it, ask a question, receive a grounded answer, and view the source pages used to generate the answer. The system should also return a fallback message when the answer is not available in the document.

4. System Architecture

The system architecture is divided into ingestion, indexing, retrieval, generation, and presentation layers. Each layer is implemented as an independent module so the project remains easy to test and extend.

Layer	Responsibility	Example Module
Ingestion	Load PDF pages and extract text	document_loader.py
Preprocessing	Clean text and remove noisy formatting	text_cleaner.py
Chunking	Split text into overlapping chunks	text_splitter.py
Embedding	Convert text into dense vectors	embeddings.py
Indexing	Store vectors and metadata	vector_store.py
Retrieval	Find relevant chunks for a question	retriever.py
RAG Pipeline	Build answer with context and sources	rag_pipeline.py
Interface	Display upload and chat experience	streamlit_app/app.py

5. Recommended Technology Stack

The recommended stack for this project is Python, PyPDF, Sentence Transformers, ChromaDB, FastAPI, Streamlit, pytest, Docker, and GitHub Actions. Sentence Transformers is used for creating embeddings. ChromaDB is used as the local persistent vector database. Streamlit is used for the chat interface. FastAPI is used to expose production-style endpoints.

6. Data Flow

The data flow begins when a PDF is uploaded. The document loader extracts text page by page. The cleaner normalizes the text. The splitter creates overlapping chunks. The embedding model converts chunks into vectors. ChromaDB stores the vectors with metadata. During question answering, the question is embedded, relevant chunks are retrieved, and the RAG pipeline builds an answer using only those chunks.

7. Chunking Strategy

The system uses metadata-aware chunking. Each chunk stores the original file name, page number, chunk number, and character count. This makes the answer explainable because the system can show exactly which page contributed to the response.

The default chunk size is 700 characters with an overlap of 120 characters. This setting provides enough context per chunk while reducing the chance of losing information across chunk boundaries. For very technical documents, the chunk size can be increased to 900 characters. For short documents, a chunk size of 500 characters may be enough.

8. Embedding Details

The default embedding model is all-MiniLM-L6-v2. It is lightweight, fast, and suitable for local semantic search experiments. The expected embedding dimension is 384. The embedding module should expose two methods: embed_texts for document chunks and embed_query for user questions.

9. Retrieval Rules

- Retrieve the top 8 chunks from the vector database during the first retrieval stage.

- Re-rank the retrieved chunks using cosine similarity or a simple relevance scoring function.

- Use the final top 3 chunks for answer generation.

- If the best relevance score is below the configured threshold, return a low-confidence fallback.

- Always include source metadata in the final response.

10. Confidence Scoring

The system assigns confidence levels based on retrieval strength. If the top relevance score is high and multiple retrieved chunks agree with the answer, confidence is High. If the evidence exists but is limited, confidence is Medium. If the system cannot find strong evidence in the document, confidence is Low.

Example threshold policy: High confidence is returned for scores above 0.75. Medium confidence is returned for scores between 0.55 and 0.75. Low confidence is returned for scores below 0.55. These values can be adjusted after testing with real documents.

11. API Specification

The backend exposes four main endpoints. The health endpoint verifies that the server is running. The upload endpoint accepts a PDF file and stores the processed chunks in the vector database. The ask endpoint accepts a question and returns an answer, confidence level, and sources. The reset endpoint clears the vector database for a fresh session.

Endpoint	Method	Purpose
/health	GET	Check whether the backend service is running.
/upload	POST	Upload and process a PDF document.
/ask	POST	Ask a question from the indexed document collection.
/reset	DELETE	Clear the current document index.

12. Expected API Response

When a question is answered successfully, the API response should contain answer, confidence, sources, and retrieved_context. The sources field must include file_name, page_number, and chunk_number. The retrieved_context field can be hidden in the UI but should be available for debugging and evaluation.

Example question: What is the default chunk size? Expected answer: The default chunk size is 700 characters with an overlap of 120 characters. Expected source: Page 3.

13. Error Handling

- If the uploaded file is not a PDF, return a clear validation error.

- If no readable text is found, return a message explaining that scanned PDFs are not supported in the first version.

- If the vector database is empty, ask the user to upload and process a document first.

- If the answer is not present, return: I could not find this information in the uploaded document.

14. Evaluation Plan

The project should be evaluated using a small set of known questions and expected answers. The goal is not only to check whether the answer sounds fluent but also whether the correct source pages are retrieved. Retrieval quality is more important than answer style in the early stages of development.

Test Question	Expected Answer Location
What is the purpose of this document?	Page 1
Which vector database is recommended?	Page 2
What is the default chunk size?	Page 3
What endpoint is used to ask questions?	Page 4
What should happen when the answer is not present?	Page 4
Which tests should be included?	Page 5

15. Testing Requirements

- test_document_loader.py should verify that PDF pages are loaded with correct metadata.

- test_text_cleaner.py should verify that extra spaces and line breaks are normalized.

- test_text_splitter.py should verify that chunks are created with metadata.

- test_vector_store.py should verify that documents can be added and retrieved.

- test_rag_pipeline.py should verify successful answers and fallback behavior.

- test_api.py should verify health, upload, ask, and reset endpoints.

16. Logging Requirements

The project should log important lifecycle events such as document upload, number of pages loaded, number of chunks generated, embedding model initialization, vector database writes, retrieval scores, and fallback responses. Logging makes the system easier to debug and more professional for production-style development.

17. Streamlit User Interface

The Streamlit interface should include a PDF upload area, a Process Document button, a chat input box, a response area, confidence label, sources section, and an expandable retrieved context section. The UI should not expose raw internal errors to the user. Instead, it should show clean messages such as Please upload a PDF first or No strong source was found.

18. Deployment Checklist

- Create a clean README with project overview, architecture, installation steps, screenshots, and sample outputs.

- Add a Dockerfile so the application can be containerized.

- Add GitHub Actions to run tests automatically on every push.

- Add .gitignore to exclude venv, cache files, uploaded PDFs, and vector database files.

- Keep sample screenshots in the screenshots folder.

- Do not upload private or sensitive documents to GitHub.

19. Resume Summary

This project demonstrates practical skills in Python engineering, natural language processing, embeddings, vector databases, RAG pipelines, backend APIs, frontend development, testing, and deployment preparation. It is suitable for a fresher ML Engineer portfolio because it combines machine learning concepts with software engineering practices.

20. Final Notes

The first version should work completely without requiring a paid LLM API. After the retrieval pipeline is stable, the answer generator can be upgraded with Gemini, OpenAI, Groq, or a local Ollama model. The most important requirement is that every answer must be grounded in retrieved document context and must show sources.

End of sample document.